Rayan Morais

Full-Stack Software Engineer

rayan_cm2021@icloud.com · linkedin.com/in/rayancmorais · Juiz de Fora, MG — Brazil · English C1
github.com/rayancmorais · cupommaniac.com.br · rayancmorais.com.br

PROFILE

Full-Stack Software Engineer with real projects in production and a focus on well-justified architecture decisions. Strong command of TypeScript, Node.js, React, and Next.js. Hands-on experience with event-driven architectures (RabbitMQ), DDD-based microservices, job queues (BullMQ/Redis), OAuth2/OIDC authentication, payment gateways (Stripe, PayPal, MercadoPago), and cloud deployment (AWS Lambda, Vercel, Railway). I value code that survives production: explicit error handling, data-driven refactors, and documentation that serves the next developer. A prior background in healthcare and law instilled zero tolerance for error and close attention to edge cases. English C1.

EXPERIENCE

Full-Stack Development Intern — Feb 2026 – Present

Vasc-Review — continuing medical education platform for vascular surgery residents and specialists

vascreview.com.br

- **Front-end web and mobile development (React Native):** new version of the platform featuring video lessons and a question bank for vascular surgery residency and board exams — app available on the App Store and Play Store, new version under active development.
- **Working in a real production environment:** shipping features, fixing bugs, and maintaining code used by medical specialists, under a formal code review process.

Freelance Full-Stack Developer — 2024 – Present

Independent projects & team collaboration

- **Collaboration with a Senior Software Engineer** (currently at AT&T) on full-stack projects — pair programming, code review, and shipping features in a real production environment.
- **End-to-end delivery of web applications** with a focus on well-justified decisions: stack selection, database modeling, deployment, and DNS configuration.
- **Diagnosing and resolving production bugs:** identifying N+1 queries, race conditions in upserts, and authentication vulnerabilities, plus third-party API integrations (affiliates, OAuth, payments, email) with explicit handling of rate limits and retries.

IT Support Analyst — 2011 – 2014

Câmara Municipal de Madre de Deus – MG (Municipal Council)

- Administered network and hardware infrastructure with high availability in an institutional setting.
- Managed the full lifecycle of technical support tickets and resolved connectivity issues.

TECHNICAL SKILLS

Languages TypeScript · JavaScript (ES6+) · Go (Golang – Intermediate) · SQL · Python (basic)

Frontend Next.js 15 · React 19 · React Native · Tailwind CSS · Framer Motion · Styled Components · Radix UI

Backend Fastify · NestJS (Controllers/Services) · Node.js · Express · Go/Gin · BullMQ · RabbitMQ · Redis (Queues/Workers) · Socket.io

Databases PostgreSQL · MongoDB · MySQL · Supabase · Prisma ORM · Meilisearch

Auth & Payments OAuth2/OIDC · NextAuth v5 · Keycloak · JWT · Stripe · PayPal · MercadoPago

Cloud & DevOps AWS Lambda · AWS SAM · AWS Secrets Manager · Docker · Vercel · Railway · GitHub Actions · Kong API Gateway

Architecture, Patterns & Quality SOLID · DDD (Domain-Driven Design) · Clean Architecture · Event-Driven Architecture (EDA) · Microservices · Serverless · Controllers, Services & Workers (NestJS) · TDD (Vitest/Jest/Testing Library) · Clean Code · Design Tokens & Dynamic Componentization

Services Cloudinary · Resend · Sentry · Anthropic Claude (API) · Pix (MercadoPago)

Design & Tooling Figma (High-Fidelity) · React Hook Form + Zod · ESLint · Prettier · Husky · Knip · Git · Postman · Linux

PROJECTS

CupomManiac — Discount Coupon Platform — 2026

cupommaniac.com.br · github.com/rayancmorais/cupomManiac

Full-stack coupon platform for the Brazilian market (similar to Méliuz), built from scratch to production on a custom domain.

- **N+1 refactor in the affiliate sync:** each cycle pulled in thousands of coupons from sources like Lomadee and Admitad, processed as individual upserts that locked the database under load. Rewritten as a batched upsertMany deduplicated by (source, externalId), indexing into Meilisearch in parallel.
- **Neon pooler + PrismaClient singleton:** Neon's free tier caps at 10 direct connections, and Next.js Server Components instantiating multiple PrismaClient instances blew past the limit under peak load. Migrated to Neon's pooled endpoint with a singleton client on the backend.
- **Proxy routes to work around cross-origin cookies:** backend (Railway) and frontend (Vercel) on different domains meant SameSite=Lax blocked cookies on fetch-based mutations. Sensitive mutations now go through Next.js proxy routes that validate the session server-side before forwarding to the backend.
- **AlertMatcherService with O(1) roundtrips:** naive alert matching would require N queries per new coupon. Rewritten as findMany → createMany (skipDuplicates) → updateMany, independent of coupon volume in the sync cycle.

- **5 affiliate sources via the Adapter Pattern:** Lomadee, Admitad, Awin, Amazon Associates, and Mercado Livre implement the same sync interface — adding a provider doesn't touch the upsert service. Automated cron jobs run sync every 3h and clean up expired coupons daily at 2am.
- **Alert system with double opt-in and algorithmic scoring (0–100):** email verification → confirmation → batch matching → alert with token-based unsubscribe; coupons ranked by discount, presence of a code, and proximity to expiration.

Stack: Next.js 15 (App Router, standalone output), TypeScript, Fastify, Prisma ORM, PostgreSQL (Neon), Redis (Upstash), BullMQ, Meilisearch, NextAuth v5, Resend, Sentry, Tailwind CSS, Vaul, Vercel, Railway

Crash Game — Full-Stack Technical Challenge (Jungle Gaming) 2026

github.com/rayancmorais/fullstack-challengeRayancm

Real-time multiplayer crash game: a multiplier climbs from 1.00x and can crash at any moment — players bet and cash out before the crash.

- **Game Service and Wallet Service communicate exclusively via RabbitMQ** — no direct HTTP between services. Rationale: Wallet must be the single source of truth for balance; synchronous HTTP would create coupling and risk inconsistency on partial failures.
- **DDD with 4 layers:** the domain layer is pure TypeScript, zero NestJS, zero Prisma — business rules testable without mocking the framework. 91 unit tests + 11 E2E scenarios covering bet→cashout→balance, bet→crash→loss, and limit validations.
- **Monetary values as BigInt cents** (PostgreSQL BIGINT, TypeScript BigInt): $1.50 = 150n — eliminates floating-point errors in financial calculations. DTOs serialize as strings to work around a JSON.stringify limitation.
- **Provably fair via HMAC-SHA256:** a server seed hash is exposed before bets open, with the seed revealed after the crash — players can independently recompute the crash point and verify there was no manipulation.
- **Kong API Gateway (DB-less)** for rate limiting and REST routing; WebSocket connects directly to the Game Service (port 4001) — Kong's DB-less mode has limitations with the HTTP→WS upgrade. CI via GitHub Actions runs parallel build, test, and lint jobs, with a Docker build-check before any merge.

Stack: NestJS, TypeScript, Bun, Next.js 15, PostgreSQL, Prisma 7, RabbitMQ, Kong, Keycloak, Socket.io, Zustand, TanStack Query, Docker, GitHub Actions

Lux Lab Brasil — Dropshipping Platform (BR + EU) 2025

luxlabbr.vercel.app · private repository, available upon request

Full-stack, multi-market dropshipping platform (Brazil and the European Union), with international suppliers, multiple payment gateways, i18n, and an AI assistant.

- **Multi-supplier with geographic routing:** a single Supplier interface (Name(), CreateOrder(), WebhookURL()) implemented by BR and EU suppliers (BigBuy, Griffati, Printful, among others); PreferredForCountry(country) returns the supplier list ranked by delivery country — adding a supplier doesn't touch the order flow. Parallel catalog sync via concurrent goroutines.
- **Region-based pricing pipeline:** normalizes supplier price into the local currency (BRL or EUR), calculates gross margin, and applies the destination country's tax rate (VAT) — three isolated, independently testable steps.
- **Multi-provider OAuth2/OIDC with PKCE:** Google, Facebook, and Apple, with state signed via OAUTH_STATE_SECRET and dual middleware (jose.jwtVerify on Next.js edge, golang-jwt on Go) mapping roles across both layers.
- **Streaming AI chat (SSE) under Lambda constraints:** API Gateway cuts connections at 29s, ruling out long-lived streams. Chat sessions with Claude (Anthropic) capped at ≤28s with an explicit timeout in the handler, documented for a future migration to a WebSocket API.
- **Production secrets security:** after credentials were accidentally exposed in .env, added gitleaks to CI to block future secret leaks, rotated credentials immediately, and migrated sensitive variables to AWS Secrets Manager.

Stack: Go, Gin, MongoDB Atlas, Next.js 15, TypeScript, Tailwind CSS v4, Radix UI, React Hook Form + Zod, AWS Lambda, AWS SAM, AWS Secrets Manager, Stripe, PayPal, MercadoPago, Cloudinary, Resend, next-intl, Claude (Anthropic), GitHub Actions

Personal Portfolio 2025

rayancmorais.com.br · github.com/rayancmorais/myportfolioficial

- **Token-based CSS design system:** the entire visual layer runs on CSS custom properties (--cy, --fg-1, --bg-1…); dark/light switching via a data-theme attribute on <html>, with no component re-render.
- **Custom contexts, no state library:** ThemeContext and LanguageContext built from scratch with createContext + useCallback, theme state persisted to localStorage — zero dependency on Redux or Zustand.
- **Smooth scroll without re-creating the loop on every render:** the original useLenis hook instantiated Lenis inside the component, so every re-render recreated the scroll loop and caused visible jank. Fixed with a module-level singleton — the instance lives outside the component and survives re-renders.
- **Fluid parallax with zero bundle cost:** an off-the-shelf parallax library would add weight just for a simple effect on the case-study screenshots. Built it by hand with getBoundingClientRect + requestAnimationFrame, checking useReducedMotion() on every animation so it never forces motion on users who've disabled it at the OS level.

Stack: React 19, TypeScript, Vite, Styled Components, Framer Motion, Lenis, i18next, Vitest, Testing Library, ESLint, Prettier, Husky, Knip, Vercel

EDUCATION

B.Sc. in Software Engineering In progress · 2025
UNINTER

B.Sc. in Nursing Completed
UNIVERSO – Juiz de Fora